

Punto 5.5 — Transacciones ambientales y buenas prácticas

Contexto del proyecto

Este punto continúa directamente M05/PROYECTO/5.4. La carpeta M05/PROYECTO/5.5 contiene la solución completa AceriaData.sln y conserva las capas y migraciones existentes. El modelo oficial no cambia en este punto; se conserva la última migración real ya existente.

Objetivo práctico

Usar TransactionScope con async, varios DbContext sobre una conexión compartida, opciones de scope y un efecto externo no transaccional.

Paso 1: Abrir el estado completo del punto

```
cd M05/PROYECTO/5.5
dotnet restore AceriaData.sln
```

La restauración se ejecuta sobre la solución local del punto. No es necesario copiar archivos desde otros puntos: la carpeta ya contiene el estado acumulativo completo.

Paso 2: Identificar los archivos principales

En este punto se trabajan principalmente estos archivos:

- src/AceriaData.Infrastructure/Repositories/TransaccionesAmbientalesM5Repositorio.cs

Los archivos se estudian dentro de su capa real. Application conserva sus abstracciones y casos de uso; Infrastructure contiene EF Core y acceso a proveedor; Console compone y ejecuta; los tests se ubican fuera de las capas de producción cuando aparecen en 5.9 y 5.11.

Paso 3: Implementar y estudiar TransaccionesAmbientalesM5Repositorio.cs

```
public async Task<TransaccionAmbientaLM5Dto> DemostrarDosContextosAsync()
{
    var options = new TransactionOptions
    {
        IsolationLevel = IsolationLevel.ReadCommitted,
        Timeout = TimeSpan.FromSeconds(30)
    };

    bool ambienteActivo;
    bool flujoAsync;
    bool promocion;
    string aislamiento;
    int ordenId;

    using (var scope = new TransactionScope(
        TransactionScopeOption.Required,
        options,
        TransactionScopeAsyncFlowOption.Enabled))
    {
        await using var connection = new SqlConnection(ObtenerConnectionString());
        await connection.OpenAsync();
        ambienteActivo = Transaction.Current is not null;
        aislamiento = Transaction.Current?.IsolationLevel.ToString() ?? "<none>";

        await using (var contextOrden = CrearContexto(connection))
        {
            var orden = CrearOrden("OF-M5-55-AMBIENT");
            contextOrden.OrdenesFabricacion.Add(orden);
            await contextOrden.SaveChangesAsync();
        }
    }
}
```

```

        ordenId = orden.Id;
    }

    await Task.Yield();
    flujoAsync = Transaction.Current is not null;

    await using (var contextPlancha = CrearContexto(connection))
    {
        contextPlancha.PlanchasAcero.Add(new PlanchaAcero
        {
            OrdenId = ordenId,
            Espesor = 10, Ancho = 1000, Largo = 2000, Peso = 150m
        });
        await contextPlancha.SaveChangesAsync();
    }

    promocion = Transaction.Current?.TransactionInformation.DistributedIdentifier != Guid.Empty;
    scope.Complete();
}

return new TransaccionAmbientaM5Dto(
    "Dos DbContext con una conexión compartida",
    ExisteOrdenYPlancha("OF-M5-55-AMBIENT"),
    ambienteActivo, flujoAsync, promocion, aislamiento,
    SqlCommandCounterInterceptor.Instance.SnapshotCommands());
}

```

Explicación línea a línea — TransaccionesAmbientalesM5Repositorio.cs

Línea 1: `public async Task<TransaccionAmbientaM5Dto> DemostrarDosContextosAsync()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `var options = new TransactionOptions` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 4: `{` → Delimita el bloque sintáctico correspondiente.

Línea 5: `IsolationLevel = IsolationLevel.ReadCommitted,` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 6: `Timeout = TimeSpan.FromSeconds(30)` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 7: `};` → Delimita el bloque sintáctico correspondiente.

Línea 9: `bool ambienteActivo;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 10: `bool flujoAsync;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 11: `bool promocion;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 12: `string aislamiento;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 13: `int ordenId;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 15: `using (var scope = new TransactionScope(` → Crea o participa en una transacción ambiental según la opción indicada.

Línea 16: `TransactionScopeOption.Required,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 17: `options,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 18: `TransactionScopeAsyncFlowOption.Enabled))` → Permite que `Transaction.Current` fluya correctamente a través de `await`.

Línea 19: `{` → Delimita el bloque sintáctico correspondiente.

Línea 20: `await using var connection = new SqlConnection(ObtenerConnectionString());` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 21: `await connection.OpenAsync();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 22: `ambienteActivo = Transaction.Current is not null;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 23: `aislamiento = Transaction.Current?.IsolationLevel.ToString() ?? "<none>";` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 25: `await using (var contextOrden = CrearContexto(connection))` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 26: `{` → Delimita el bloque sintáctico correspondiente.

Línea 27: `var orden = CrearOrden("OF-M5-55-AMBIENT");` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 28: `contextOrden.OrdenesFabricacion.Add(orden);` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 29: `await contextOrden.SaveChangesAsync();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 30: `ordenId = orden.Id;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 31: `}` → Delimita el bloque sintáctico correspondiente.

Línea 33: `await Task.Yield();` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 34: `flujoAsync = Transaction.Current is not null;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 36: `await using (var contextPlancha = CrearContexto(connection))` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 37: `{` → Delimita el bloque sintáctico correspondiente.

Línea 38: `contextPlancha.PlanchasAcero.Add(new PlanchaAcero` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 39: `{` → Delimita el bloque sintáctico correspondiente.

Línea 40: `OrdenId = ordenId,` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 41: `Espesor = 10, Ancho = 1000, Largo = 2000, Peso = 150m` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 42: `});` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 43: `await contextPlancha.SaveChangesAsync();` → Envía al proveedor los cambios rastreados y aplica las reglas de transacción/concurrencia configuradas.

Línea 44: `}` → Delimita el bloque sintáctico correspondiente.

Línea 46: `promocion = Transaction.Current?.TransactionInformation.DistributedIdentifier != Guid.Empty;` → Comprueba si la transacción fue promocionada a una transacción distribuida.

Línea 47: `scope.Complete();` → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.

Línea 48: `}` → Delimita el bloque sintáctico correspondiente.

Línea 50: `return new TransaccionAmbientaIM5Dto(` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 51: `"Dos DbContext con una conexión compartida",` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 52: `ExisteOrdenYPlancha("OF-M5-55-AMBIENT"),` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 53: `ambienteActivo, flujoAsync, promocion, aislamiento,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 54: `SqlCommandCounterInterceptor.Instance.SnapshotCommands();` → Recupera los comandos SQL realmente interceptados durante la operación.

Línea 55: `}` → Delimita el bloque sintáctico correspondiente.

Paso 4: Implementar y estudiar `TransaccionesAmbientalesM5Repositorio.cs`

```
public OpcionesTransactionScopeM5Dto DemostrarOpcionesDeScope()
{
    string outerId;
    bool requiredMisma;
    bool requiresNewOtra;
    bool suppressSinAmbiente;
    string aislamiento;

    using (var outer = new TransactionScope())
    {
        var outerTransaction = Transaction.Current
            ?? throw new InvalidOperationException("No se creó la transacción ambiental.");
        outerId = outerTransaction.TransactionInformation.LocalIdentifier;
        aislamiento = outerTransaction.IsolationLevel.ToString();

        using (var required = new TransactionScope(TransactionScopeOption.Required))
        {
            requiredMisma = Transaction.Current?.TransactionInformation.LocalIdentifier == outerId;
            required.Complete();
        }

        using (var requiresNew = new TransactionScope(TransactionScopeOption.RequiresNew))
        {
            requiresNewOtra = Transaction.Current?.TransactionInformation.LocalIdentifier != outerId;
            requiresNew.Complete();
        }

        using (var suppressed = new TransactionScope(TransactionScopeOption.Suppress))
        {
            suppressSinAmbiente = Transaction.Current is null;
            suppressed.Complete();
        }
        outer.Complete();
    }

    return new OpcionesTransactionScopeM5Dto(
        requiredMisma, requiresNewOtra, suppressSinAmbiente,
        aislamiento, TransactionManager.DefaultTimeout.TotalSeconds);
}
```

Explicación línea a línea — `TransaccionesAmbientalesM5Repositorio.cs`

Línea 1: `public OpcionesTransactionScopeM5Dto DemostrarOpcionesDeScope()` → Declara la operación pública que ejecutará o verificará este comportamiento.

Línea 2: `{` → Delimita el bloque sintáctico correspondiente.

Línea 3: `string outerId;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 4: `bool requiredMisma;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 5: `bool requiresNewOtra;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 6: `bool suppressSinAmbiente;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 7: `string aislamiento;` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 9: `using (var outer = new TransactionScope())` → Crea o participa en una transacción ambiental según la opción indicada.

Línea 10: `{` → Delimita el bloque sintáctico correspondiente.

Línea 11: `var outerTransaction = Transaction.Current` → Calcula o conserva un valor que forma parte de la preparación, ejecución o verificación del escenario.

Línea 12: `?? throw new InvalidOperationException("No se creó la transacción ambiental.");` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 13: `outerId = outerTransaction.TransactionInformation.LocalIdentifier;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 14: `aislamiento = outerTransaction.IsolationLevel.ToString();` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 16: `using (var required = new TransactionScope(TransactionScopeOption.Required))` → Crea o participa en una transacción ambiental según la opción indicada.

Línea 17: `{` → Delimita el bloque sintáctico correspondiente.

Línea 18: `requiredMisma = Transaction.Current?.TransactionInformation.LocalIdentifier == outerId;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 19: `required.Complete();` → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.

Línea 20: `}` → Delimita el bloque sintáctico correspondiente.

Línea 22: `using (var requiresNew = new TransactionScope(TransactionScopeOption.RequiresNew))` → Crea o participa en una transacción ambiental según la opción indicada.

Línea 23: `{` → Delimita el bloque sintáctico correspondiente.

Línea 24: `requiresNewOtra = Transaction.Current?.TransactionInformation.LocalIdentifier != outerId;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 25: `requiresNew.Complete();` → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.

Línea 26: `}` → Delimita el bloque sintáctico correspondiente.

Línea 28: `using (var suppressed = new TransactionScope(TransactionScopeOption.Suppress))` → Crea o participa en una transacción ambiental según la opción indicada.

Línea 29: `{` → Delimita el bloque sintáctico correspondiente.

Línea 30: `suppressSinAmbiente = Transaction.Current is null;` → Asigna el valor necesario para preparar o registrar el estado de esta operación.

Línea 31: `suppressed.Complete();` → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.

Línea 32: `}` → Delimita el bloque sintáctico correspondiente.

Línea 33: `outer.Complete();` → Marca/confirma la unidad transaccional cuando todas las operaciones requeridas han terminado correctamente.

Línea 34: `}` → Delimita el bloque sintáctico correspondiente.

Línea 36: `return new OpcionesTransactionScopeM5Dto(` → Devuelve la evidencia recopilada por el escenario para que el caso de uso o el test pueda comprobarla.

Línea 37: `requiredMisma, requiresNewOtra, suppressSinAmbiente,` → Forma parte de la implementación concreta del comportamiento explicado en este paso.

Línea 38: `aislamiento, TransactionManager.DefaultTimeout.TotalSeconds);` → Ejecuta esta instrucción como parte del flujo del ejemplo.

Línea 39: `}` → Delimita el bloque sintáctico correspondiente.

Paso 5: Compilar y ejecutar

```
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
dotnet ef migrations has-pending-model-changes --project src/AceriaData.Infrastructure --startup-
project src/AceriaData.Console --configuration Release
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release --no-build
```

La ejecución debe finalizar con `5.5 OK`. Si una comprobación interna falla, la aplicación lanza una excepción y el punto no se considera correctamente reproducido.

Diagnóstico técnico

La práctica se considera correcta cuando se observan estas condiciones:

- `TransactionScopeAsyncFlowOption.Enabled` conserva `Transaction.Current` tras `await`.
- Los dos `DbContext` usan una misma `SqlConnection` abierta para no depender de promoción distribuida.
- `Required`, `RequiresNew` y `Suppress` se distinguen por su `Transaction.Current`.
- Omitir `Complete` revierte SQL.
- El efecto externo simulado permanece porque no participa en `System.Transactions`.

Reto resuelto y ampliación

Reto: Ejecuta el escenario con `Suppress` alrededor de una operación y verifica qué queda fuera de la transacción ambiental.

Para resolverlo, conserva la misma regla del módulo: primero define qué resultado funcional debe mantenerse; después captura SQL, número de comandos, tracking, historial de migraciones o estado transaccional según corresponda. No uses una medición temporal aislada como sustituto de la evidencia técnica.

Errores comunes

- Asumir que `ReadCommitted` permite lecturas sucias.
- Afirmar que `Snapshot` elimina todos los bloqueos y conflictos.
- Abrir varios recursos durables sin considerar posible promoción a transacción distribuida.
- Esperar que HTTP, colas o archivos se reviertan con `TransactionScope`.

Cierre acumulativo

1. Ejecuta `dotnet build` en la solución del punto.
2. Ejecuta la aplicación y comprueba el marcador `5.5 OK`.
3. Conserva las migraciones existentes; solo 5.2 añade el cambio de modelo oficial de M5.
4. Comprueba que Application continúa sin depender de Entity Framework Core ni de Infrastructure.
5. Antes de avanzar, relaciona el resultado con el SQL, la transacción, el historial o la prueba que lo demuestra.

Resultado esperado

Al finalizar el punto 5.5, AceriaData debe compilar, ejecutarse sobre SQL Server LocalDB y demostrar de forma observable el comportamiento descrito. El marcador final es:

5.5 OK

Conexión con el siguiente punto

El siguiente estado es `5.6` y parte directamente de este proyecto completo.
